STA 651 Final Project (Fall 2026)

Building an R Package for Negative Binomial and Zero-Inflated Negative Binomial Regression

Instructor: Dr. Donghyung Lee

October 26, 2026

Table of contents

1 Overview and Key Dates **2**

2 Statistical Background **3**

3 Required Algorithms **3**

4 R Package Requirements **5**

5 Benchmarking Requirements **7**

6 Report and GitHub Pages Requirements **8**

7 Submission **9**

8 Using Claude Code **9**

9 Oral Walkthrough (November 2–6, 2026) **10**

10 Grading Rubric (100 points + 5 bonus) **10**

11 Suggested One-Week Schedule **10**

12 References **11**

Appendix A: Worked Example with `bioChemists` **12**

1 Overview and Key Dates

In this project you will build, test, document, and publish an **R package** that fits

1. **negative binomial (NB) regression**, similar to `MASS::glm.nb()`, and
2. **zero-inflated negative binomial (ZINB) regression**, similar to `pscl::zeroinfl(..., dist = "negbin")`.

The core of the project is that **you implement the estimation yourself**, using iteratively reweighted least squares (IRLS). You are **not** allowed to call `optim()` or any other general-purpose optimizer. You will then show that your functions reproduce the results of the standard R functions, analyze a real data set, and write a short report that is submitted as a PDF and also published on the web through GitHub Pages.

This is an **individual** project. You are expected (and encouraged) to use **Claude Code** as a programming assistant; see Section 8 for the rules. A separate document, *STA 651 Final Project: Setup Guide*, walks you through installing R, VS Code, Git/GitHub, Quarto, and Claude Code.

Table 1: Key dates

Item	Date / Deadline
Project assigned	Monday, October 26, 2026
Submission deadline	**Monday, November 2, 2026, 11:59 PM (Eastern)**
What to submit on Canvas	(1) PDF report **and** (2) link to your GitHub repository
Oral walkthrough	Monday, November 2 – Friday, November 6, 2026
Walkthrough length	10–15 minutes, one-on-one with Dr. Lee

! Schedule your oral walkthrough

Every student **must** book a 10–15 minute meeting with Dr. Lee during **November 2–6, 2026** using the sign-up method announced on Canvas. Book your slot **before** the November 2 deadline. A project without a completed walkthrough will not receive a final grade.

Late policy. The GitHub repository is evaluated as of the last commit made **on or before** 11:59 PM on November 2. Commits made after the deadline will not be graded unless you have prior approval.

1.1 Learning objectives

By completing this project you will be able to

- derive and implement IRLS (Fisher scoring) for a GLM with a non-canonical variance function;
- estimate a dispersion parameter by Newton–Raphson and alternate it with IRLS;
- explain why the ZINB model is **not** a GLM, and fit it with an **EM algorithm whose M-steps are IRLS fits**;
- compute standard errors from the (observed or expected) information matrix;
- build an installable, documented, and tested R package and publish it on GitHub;
- check your own software against established reference implementations;
- choose between Poisson, NB, and ZINB models and interpret the results.

2 Statistical Background

2.1 Negative binomial regression

Let Y_i be a count and x_i a p-vector of covariates, $i = 1, \ldots, n$. The NB2 model assumes

$$P(Y_i = y) \;=\; \frac{\Gamma(y+\theta)}{\Gamma(\theta)\, y!} \left(\frac{\theta}{\theta+\mu_i}\right)^{\theta} \left(\frac{\mu_i}{\theta+\mu_i}\right)^{y}, \qquad y = 0, 1, 2, \ldots$$

with log link $\log \mu_i = \eta_i = x_i^\top \beta + o_i$, where o_i is an optional offset (e.g., log exposure). Then $E(Y_i) = \mu_i$ and $\mathrm{Var}(Y_i) = \mu_i + \mu_i^2/\theta$, so $\theta > 0$ controls overdispersion. As $\theta \to \infty$ the model reduces to the Poisson. The log-likelihood, with optional prior weights a_i (all $a_i = 1$ for an ordinary fit), is

$$\ell(\beta, \theta) = \sum_{i=1}^{n} a_i \Big[\log\Gamma(y_i + \theta) - \log\Gamma(\theta) - \log y_i! + \theta \log\theta + y_i \log\mu_i - (\theta + y_i)\log(\theta + \mu_i) \Big].$$

This is the parameterization used by `MASS::glm.nb()`. Its `theta` is the same θ.

2.2 Zero-inflated negative binomial regression

The ZINB model is a two-component mixture. With probability π_i the observation is a **structural zero**. With probability $1 - \pi_i$ it comes from an $\mathrm{NB}(\mu_i, \theta)$ distribution, which can itself produce zeros:

$$P(Y_i = 0) = \pi_i + (1 - \pi_i)\, f_{\mathrm{NB}}(0; \mu_i, \theta), \qquad P(Y_i = y) = (1 - \pi_i)\, f_{\mathrm{NB}}(y; \mu_i, \theta),\ y \geq 1,$$

where $f_{\mathrm{NB}}(0; \mu, \theta) = \{\theta/(\theta+\mu)\}^\theta$, $\log \mu_i = x_i^\top \beta + o_i$ (count model) and $\mathrm{logit}\, \pi_i = z_i^\top \gamma$ (zero-inflation model). The covariates z_i may differ from x_i. The marginal mean is $E(Y_i) = (1 - \pi_i)\mu_i$.

The observed-data log-likelihood is

$$\ell(\beta, \gamma, \theta) = \sum_{i:\, y_i = 0} \log\big[\pi_i + (1 - \pi_i) f_{\mathrm{NB}}(0; \mu_i, \theta)\big] + \sum_{i:\, y_i > 0} \Big[\log(1 - \pi_i) + \log f_{\mathrm{NB}}(y_i; \mu_i, \theta) \Big].$$

Because of the mixture, ℓ is **not** the log-likelihood of an exponential-family GLM, so IRLS cannot be applied to it directly. `pscl::zeroinfl()` maximizes it with `optim()` (BFGS). **You will maximize it with EM, using IRLS inside each M-step.**

3 Required Algorithms

3.1 NB regression: IRLS for β alternated with Newton–Raphson for θ

(a) IRLS for β with θ held fixed. For a GLM with log link and variance $V(\mu) = \mu + \mu^2/\theta$, Fisher scoring is equivalent to repeated weighted least squares with working weights and working response

$$w_i = a_i \frac{(\partial \mu_i/\partial \eta_i)^2}{V(\mu_i)} = a_i \frac{\theta\,\mu_i}{\theta+\mu_i}, \qquad z_i = (\eta_i - o_i) + \frac{y_i-\mu_i}{\mu_i},$$

$$\beta^{(t+1)} = (X^\top W X)^{-1} X^\top W z, \qquad W = \mathrm{diag}(w_1,\dots,w_n).$$

Iterate until the change in β (or in the deviance or log-likelihood) is below a tolerance. Use `solve()`, `qr()` or `chol()` for the weighted least-squares step. Do not use `lm()`, `lm.wfit()` or `glm.fit()`.

(b) Newton–Raphson for θ with β held fixed. The score and second derivative are

$$U(\theta) = \sum_i a_i \Big[\psi(y_i+\theta) - \psi(\theta) + \log\theta + 1 - \log(\theta+\mu_i) - \frac{y_i+\theta}{\theta+\mu_i}\Big],$$

$$H(\theta) = \sum_i a_i \Big[\psi'(y_i+\theta) - \psi'(\theta) + \frac{1}{\theta} - \frac{2}{\theta+\mu_i} + \frac{y_i+\theta}{(\theta+\mu_i)^2}\Big],$$

where ψ and ψ' are `digamma()` and `trigamma()`. Update $\theta \leftarrow \theta - U(\theta)/H(\theta)$. Updating $\phi = \log\theta$ is **more stable** because it keeps $\theta > 0$. Use $U_\phi = \theta U$ and $H_\phi = \theta^2 H + \theta U$.

(c) Outer loop. Start from a Poisson IRLS fit (or $\log\bar{y}$ for the intercept) and an initial θ, for example a moment estimate. Then alternate (a) and (b) until the log-likelihood changes by less than a tolerance such as 10^{-8}.

(d) Standard errors. At convergence, report $\widehat{\mathrm{Var}}(\hat\beta) = (X^\top \hat W X)^{-1}$ (treating θ as known, as `glm.nb()` does) and $\mathrm{SE}(\hat\theta) = \{-H(\hat\theta)\}^{-1/2}$.

3.2 ZINB regression: EM algorithm with IRLS M-steps

Introduce a latent indicator $D_i = 1$ if observation i is a structural zero. Given current values (β, γ, θ):

E-step. Compute the posterior probability of being a structural zero:

$$\tau_i = P(D_i = 1 \mid y_i) = \begin{cases} \dfrac{\pi_i}{\pi_i + (1-\pi_i)\, f_{\mathrm{NB}}(0;\mu_i,\theta)}, & y_i = 0, \\ 0, & y_i > 0. \end{cases}$$

M-step (zero part). Maximize $\sum_i [\tau_i \log\pi_i + (1-\tau_i)\log(1-\pi_i)]$ over γ. This is a logistic regression with a *fractional* response τ_i. Fit it by IRLS with $w_i = \pi_i(1-\pi_i)$ and $z_i = z_i^\top\gamma + (\tau_i - \pi_i)/\{\pi_i(1-\pi_i)\}$.

M-step (count part). Maximize $\sum_i (1-\tau_i)\log f_{\mathrm{NB}}(y_i;\mu_i,\theta)$ over (β,θ). This is exactly the NB fit of Section 3.1 with **prior weights** $a_i = 1-\tau_i$, so reuse your NB code.

Convergence. Repeat E and M steps until the **observed-data** log-likelihood $\ell(\beta,\gamma,\theta)$ changes by less than a tolerance (e.g., 10^{-8}). EM must never decrease ℓ. Checking this at each iteration is an excellent debugging tool.

Starting values. Use the NB fit for (β,θ). For γ, use an intercept-only value or your own IRLS logistic regression of $I(y_i = 0)$ on z_i.

Standard errors. EM does not produce standard errors. Compute the observed information matrix of the **observed-data** log-likelihood with respect to $(\beta, \gamma, \log\theta)$ at the MLE and invert it. Either approach is acceptable:

- **Analytic Hessian** (preferred; +5 bonus points), or
- **Numerical Hessian** of *your own* log-likelihood function using `numDeriv::hessian()`. This is differentiation, not optimization, so it is allowed.

Report $\log\theta$ with its standard error, as `zeroinfl()` does, and also θ.

> **i** Feasibility check
>
> An instructor reference implementation that follows exactly the steps above reproduces `glm.nb()` and `zeroinfl()` on the `bioChemists` data (Appendix A). The coefficients agree to about 10^{-5} and the log-likelihoods are identical, with EM converging in about 120 iterations (under one second).

3.3 Allowed and prohibited functions

Table 2: Allowed and prohibited functions

Allowed in your fitting code	**Prohibited** in your fitting code
`solve`, `qr`, `chol`, `crossprod`, `%*%`	`optim`, `optimize`/`optimise`, `nlm`, `nlminb`, `uniroot`
`lgamma`, `digamma`, `trigamma`, `dnbinom`, `plogis`, `qlogis`	`glm`, `glm.fit`, `lm`, `lm.fit`, `lm.wfit`
`model.frame`, `model.matrix`, `model.response`, `model.offset`	`MASS::glm.nb`, `MASS::theta.ml`, any `MASS` fitting code
`Formula` package for two-part formulas	`pscl::zeroinfl`, `countreg`, `glmmTMB`, `gamlss`, `VGAM`
`numDeriv::hessian` (for standard errors only)	`optimx`, `maxLik`, `stats4::mle`, `bbmle`, or any other optimizer

The reference functions (`glm`, `glm.nb`, `zeroinfl`) **may and should** be used in your **tests, benchmark scripts, and report**. They must not be used inside your fitting functions.

4 R Package Requirements

4.1 Installation must work

Your package must install on a clean machine with

```
# install.packages("devtools")
devtools::install_github("<your-github-username>/<your-repo-name>")
```

The instructor will run this command in a fresh R session. **If the package does not install, the implementation portion of the grade cannot be earned.** To make sure it installs:

- list every package you `import` in the `Imports:` field of `DESCRIPTION`, and packages used only in tests/vignettes (`MASS`, `pscl`, `AER`, `testthat`, ...) in `Suggests:`;
- `devtools::check()` (equivalently `R CMD check`) must finish with **0 errors and 0 warnings**. Explain any remaining NOTEs in the README;
- add the GitHub Actions workflow `usethis::use_github_action("check-standard")` and show its passing badge in the README;
- before submitting, test the `install_github()` command yourself in a **new** R session.

4.2 Package name and required functions

Choose a valid package name that is **not** already on CRAN (e.g., `nbzinb` followed by your last name, such as `nbzinbSmith`). Your package must export at least:

Table 3: Required user-facing functions

Function	Purpose
`nbreg(formula, data, offset, control)`	NB regression fitted by IRLS + Newton–Raphson for θ
`zinbreg(formula, data, offset, control)`	ZINB regression via EM with IRLS M-steps, using a two-part formula (see below)
`nbreg_control()`, `zinbreg_control()`	Tolerances, maximum iterations, `trace` option

The `zinbreg()` formula has the form `y ~ x1 + x2 | z1 + z2`: count-model covariates before the vertical bar and zero-model covariates after it. Without `|`, use the same covariates in both parts, as `zeroinfl()` does.

Each fitting function returns an S3 object (class `"nbreg"` or `"zinbreg"`) that contains at least: coefficients, variance–covariance matrix, $\hat{\theta}$ and its SE, log-likelihood, number of iterations, a convergence flag, fitted values, and the matched call.

Required S3 methods for both classes: `print`, `summary` (with a coefficient table of estimates, SEs, z values, and p-values, plus a `print.summary` method), `coef`, `vcov`, `logLik` (with a `df` attribute so that `AIC()` and `BIC()` work), `fitted`, `residuals` (response and Pearson), `nobs`, and `predict` (with `newdata`). For `zinbreg`, `predict` should support `type = "response"` ($(1-\pi)\mu$), `"count"` (μ), and `"zero"` (π).

4.3 Documentation, tests, and website

- **roxygen2** documentation for every exported function, with runnable `@examples`.
- **testthat** tests in `tests/testthat/` that (i) compare `nbreg()` with `MASS::glm.nb()` and `zinbreg()` with `pscl::zeroinfl()` within the tolerances in Table 4, and (ii) check basic behavior (e.g., an offset, a single-covariate model, and informative errors for bad input). Use `skip_if_not_installed("pscl")` for reference packages.
- A **README.md** with installation instructions, a short usage example, and the CI badge.
- A **pkgdown website** published with **GitHub Pages** (Section 6).
- Handle edge cases sensibly: warn if $\hat{\theta}$ becomes very large (Poisson limit) or if the algorithm hits `maxit` without converging.

5 Benchmarking Requirements

Benchmark your functions against the reference functions on **at least two data sets**: `pscl::bioChemists` and `AER::NMES1988` (Section 5.2). For each data set and model, report a table that shows the two sets of estimates side by side and their maximum absolute difference.

Table 4: Required agreement (maximum absolute difference)

Quantity	NB vs `glm.nb()`	ZINB vs `zeroinfl()`
Coefficients	$\leq 10^{-5}$	$\leq 10^{-3}$
Standard errors	$\leq 10^{-5}$	$\leq 10^{-3}$
θ (NB) / $\log\theta$ (ZINB)	$\leq 10^{-4}$	$\leq 10^{-3}$
Log-likelihood	$\leq 10^{-4}$	$\leq 10^{-3}$

Tips: run the reference functions with tight tolerances, for example `glm.nb(..., control = glm.control(epsilon = 1e-10, maxit = 100))` and `zeroinfl(..., control = zeroinfl.control(reltol = 1e-12))`. Note that `zeroinfl()` labels its coefficients `count_*` and `zero_*`. If your log-likelihood is **higher** than the reference value, report it. That is not an error.

Also report **computation time** (e.g., with `system.time()` or the `bench` package) for your functions and the reference functions. Your code does not need to be faster.

5.1 Simulation study

Write a small simulation to show that `zinbreg()` recovers known parameters. Simulate at least $R = 100$ data sets with $n = 500$ from a ZINB model with one or two covariates in each part and known (β, γ, θ). Report the bias of each estimate, the mean of the estimated SEs vs. the Monte Carlo SD, and the empirical coverage of 95% Wald intervals. Set a seed so that the results are reproducible.

5.2 Data analysis: NMES1988

Analyze `NMES1988` from the **AER** package. These are physician office visits for 4,406 individuals aged 66 and over in the US National Medical Expenditure Survey, 1987–88; see Deb and Trivedi (1997) and Zeileis, Kleiber, and Jackman (2008). Use

- **response:** `visits` (number of physician office visits);
- **count-model covariates:** `hospital`, `health`, `chronic`, `gender`, `school`, `insurance`;
- **zero-model covariates:** at minimum `hospital`, `chronic`, `gender`, `school`, `insurance`. You may justify a different choice.

Using **your package**, fit NB and ZINB models (and a Poisson model with `glm()` for reference). Your analysis must include:

1. a descriptive summary that shows evidence of overdispersion and of more zeros than a Poisson model predicts;
2. model comparison (log-likelihood, AIC, BIC, and observed vs. expected number of zeros);
3. an interpretation of the main effects as **incidence rate ratios** (count part) and **odds ratios** (zero part), with 95% confidence intervals, in plain language;
4. a short discussion of which model you prefer and why.

Appendix A shows a complete worked example of this analysis on `bioChemists`, fitted with the reference functions. Use it as a template, but **do not copy its text**. Your report must be about NMES1988 and must use your own package.

6 Report and GitHub Pages Requirements

6.1 Report content (6–10 pages, excluding references and appendix)

1. **Introduction.** Count data, overdispersion, and excess zeros; goals of the package.
2. **Methods.** The NB and ZINB models, your IRLS, Newton–Raphson, and EM algorithms (with equations), how standard errors are computed, and convergence criteria.
3. **The R package.** Design, main functions, and S3 methods, with a short usage example and installation instructions.
4. **Benchmarking.** Agreement tables and timing for both data sets.
5. **Simulation study.** Results table and a brief discussion.
6. **Data analysis.** NMES1988 analysis (Section 5.2).
7. **Discussion.** Limitations and possible extensions.
8. **AI-use statement.** How you used Claude Code (Section 8): what you asked it to do, what you verified yourself, and one example where you had to correct or reject its output.
9. **References.**

The report must be written in **Quarto** and be **fully reproducible**: all numbers, tables, and figures are generated by code chunks that call your installed package.

6.2 Two output formats from one source

Write the report **once** as `vignettes/articles/report.qmd`. Create it with `usethis::use_article("report")` and rename the file to `.qmd`, or ask Claude Code to set this up. From that single source:

- render the **PDF** with `quarto render vignettes/articles/report.qmd --to pdf`, commit the PDF to the repository as `report/report.pdf`, and **upload the same PDF to Canvas**;
- build the **website** with `pkgdown::build_site()`. The report appears as an article, alongside the function reference generated from your roxygen documentation.

Publish the website with GitHub Pages. The simplest approach is to build the site locally into the `docs/` folder, commit it, and in GitHub go to *Settings* → *Pages* → *Build and deployment* → *Deploy from a branch* → ***main*** / ***/docs***. Your site will be at `https://<username>.github.io/<repo-name>/`. **The repository must be public** so that Pages works and the instructor can install the package. Put the website link in the README and in the repository's *About* box.

6.3 Expected repository layout

```
<repo-name>/
|-- DESCRIPTION, NAMESPACE, LICENSE, README.md, _pkgdown.yml
|-- R/                      # nbreg.R, zinbreg.R, methods.R, utils.R ...
|-- man/                    # generated by roxygen2 (do not edit by hand)
|-- tests/testthat/         # test-nbreg.R, test-zinbreg.R ...
|-- vignettes/articles/     # report.qmd (single source of the report)
```

```
|-- inst/benchmark/        # benchmark.R, simulation.R (+ saved results)
|-- report/report.pdf      # rendered PDF report
|-- docs/                  # pkgdown website (GitHub Pages)
`-- .github/workflows/     # R-CMD-check.yaml
```

7 Submission

On **Canvas**, by **Monday, November 2, 2026, 11:59 PM (Eastern)**, submit:

1. your **PDF report**, and
2. the **URL of your public GitHub repository** (also include the GitHub Pages URL in the submission comment).

Pre-submission checklist

- ☐ `devtools::install_github("<user>/<repo>")` works in a fresh R session
- ☐ `devtools::check()`: 0 errors, 0 warnings; CI badge is green
- ☐ `devtools::test()` passes, including the comparisons with `glm.nb()` and `zeroinfl()`
- ☐ No prohibited functions in `R/` (search for `optim`, `glm`, `zeroinfl`, ...)
- ☐ Report renders to PDF; `report/report.pdf` is committed and uploaded to Canvas
- ☐ GitHub Pages site is live and contains the report and the function reference
- ☐ Commit history shows your progress over the week (not a single final commit)
- ☐ Oral walkthrough slot with Dr. Lee is booked for November 2–6

8 Using Claude Code

You are **expected** to use Claude Code for this project. Learning to work productively with an AI coding assistant is part of the goal. The rules are simple:

- **You are responsible for every line.** Anything in your repository is treated as your work. You must be able to explain it in the oral walkthrough.
- **Understand the statistics first.** Read Section 2.1 through Section 3.2 before you ask Claude Code to write code. Check that the code it writes implements *these* equations, for example the IRLS weights, the E-step τ_i, and the θ score.
- **Enforce the rules.** Tell Claude Code explicitly that `optim()` and the other prohibited functions in Table 2 are not allowed. Put this rule in your project's `CLAUDE.md` file, which Claude Code reads every session. Check `R/` yourself before you submit.
- **Commit often.** Small commits with clear messages, made throughout the week, document your process.
- **Disclose.** Include the AI-use statement in your report (Section 6.1, item 8).

Good prompts are specific. For example: *"Implement `nbreg()` in `R/nbreg.R` using the IRLS update and the Newton–Raphson update for log(theta) exactly as written in Section 3.1 of the project description (pasted below). Do not use optim, glm, or glm.fit. Then write a testthat test that compares the result with MASS::glm.nb on pscl::bioChemists."*

9 Oral Walkthrough (November 2–6, 2026)

Each meeting with Dr. Lee lasts 10–15 minutes. Bring a laptop with your package installed. Expect to:

1. **Demo (about 3 minutes).** Install your package from GitHub, fit both models, and show the agreement with `glm.nb()` and `zeroinfl()`.
2. **Code walkthrough (about 5 minutes).** Show where the IRLS weights and working response are computed, how θ is updated, and how the E-step and M-steps are implemented.
3. **Questions (about 5 minutes).** For example: Why can't ZINB be fitted by IRLS alone? What happens to your algorithm as $\theta \to \infty$? How did you compute the ZINB standard errors? Which zero-model covariates mattered in NMES1988, and how do you interpret them? What was one mistake Claude Code made and how did you catch it?

The walkthrough is graded (Section 10). If you cannot explain your own implementation, the implementation scores may also be reduced.

10 Grading Rubric (100 points + 5 bonus)

Table 5: Grading rubric

Component	Points	Criteria
Installation & reproducibility	10	Installs with `install_github`; `check()` clean; CI passes; report reproducible
NB implementation	20	IRLS + Newton–Raphson coded correctly; agreement with `glm.nb()` per Table 4
ZINB implementation	25	EM with IRLS M-steps; correct SEs; agreement with `zeroinfl()` per Table 4
Package quality	10	S3 methods, roxygen docs, meaningful tests, README, pkgdown site
Simulation & data analysis	10	Simulation study; NMES1988 analysis with correct interpretation
Report	10	Clear methods with equations, complete sections, PDF + GitHub Pages
Oral walkthrough	15	Demonstrates understanding of code and methods
Bonus	+5	Analytic Hessian for ZINB standard errors
Total	**100**	

11 Suggested One-Week Schedule

Table 6: Suggested schedule

Day	Goal
Mon, Oct 26	Complete the Setup Guide. Create the GitHub repo and the package skeleton (`usethis::create_package()`). Write `CLAUDE.md`. Read Sections 2–3 of this document.

Day	Goal
Tue, Oct 27	Implement `nbreg()` (IRLS + θ update). Test it against `glm.nb()`. Commit.
Wed, Oct 28	Implement `zinbreg()` (EM with IRLS M-steps) and check that the log-likelihood never decreases. Test against `zeroinfl()`.
Thu, Oct 29	Standard errors; S3 methods; `predict`; roxygen documentation; `devtools::check()`.
Fri, Oct 30	GitHub Actions CI; benchmarks on both data sets; simulation study.
Sat–Sun, Oct 31 – Nov 1	NMES1988 analysis; write the report; build the pkgdown site; enable GitHub Pages.
Mon, Nov 2	Fresh-session install test; final check; submit PDF + repo link on Canvas by 11:59 PM.

12 References

- Cameron, A. C. and Trivedi, P. K. (2013). *Regression Analysis of Count Data* (2nd ed.). Cambridge University Press.
- Deb, P. and Trivedi, P. K. (1997). Demand for medical care by the elderly: A finite mixture approach. *Journal of Applied Econometrics*, 12(3), 313–336.
- Dempster, A. P., Laird, N. M. and Rubin, D. B. (1977). Maximum likelihood from incomplete data via the EM algorithm. *JRSS-B*, 39(1), 1–38.
- Lambert, D. (1992). Zero-inflated Poisson regression, with an application to defects in manufacturing. *Technometrics*, 34(1), 1–14.
- Long, J. S. (1997). *Regression Models for Categorical and Limited Dependent Variables*. Sage.
- McCullagh, P. and Nelder, J. A. (1989). *Generalized Linear Models* (2nd ed.). Chapman & Hall.
- Venables, W. N. and Ripley, B. D. (2002). *Modern Applied Statistics with S* (4th ed.). Springer. (`MASS::glm.nb`)
- Wickham, H. and Bryan, J. (2023). *R Packages* (2nd ed.). O'Reilly. https://r-pkgs.org
- Zeileis, A., Kleiber, C. and Jackman, S. (2008). Regression models for count data in R. *Journal of Statistical Software*, 27(8), 1–25.

Appendix A: Worked Example with `bioChemists`

This appendix shows the kind of analysis expected in your report. It uses the **reference** functions; your report must use **your own** package on NMES1988.

A.1 Data

The `bioChemists` data (Long, 1997), included in the **pscl** package, record the number of articles (`art`) published in the last three years of the PhD by 915 biochemistry graduate students. The covariates are gender (`fem`), marital status (`mar`), number of children under age 6 (`kid5`), prestige of the PhD program (`phd`), and the number of articles published by the student's mentor (`ment`).

```
library(MASS)
library(pscl)
data("bioChemists", package = "pscl")
d <- bioChemists
y <- d$art
c(n = length(y), mean = mean(y), variance = var(y),
  prop_zero = mean(y == 0), poisson_prop_zero = dpois(0, mean(y)))
```

```
#>                n             mean         variance        prop_zero
#>       915.0000000        1.6928962        3.7097416        0.3005464
#> poisson_prop_zero
#>         0.1839859
```

The variance (3.71) is more than twice the mean (1.69), which indicates **overdispersion**. 30.1% of the students published no article, while a Poisson distribution with the same mean predicts only 18.4%. Both features motivate NB and ZINB models.

```
tab <- table(factor(pmin(y, 12), levels = 0:12))
bp <- barplot(tab, col = "grey80", border = "grey40", ylim = c(0, 300),
              xlab = "Articles (12 = 12 or more)", ylab = "Frequency")
points(bp, length(y) * dpois(0:12, mean(y)), pch = 19, col = "firebrick")
legend("topright", c("Observed", "Poisson expected"), pch = c(22, 19),
       pt.bg = c("grey80", NA), pt.cex = c(2, 1),
       col = c("grey40", "firebrick"), bty = "n")
```

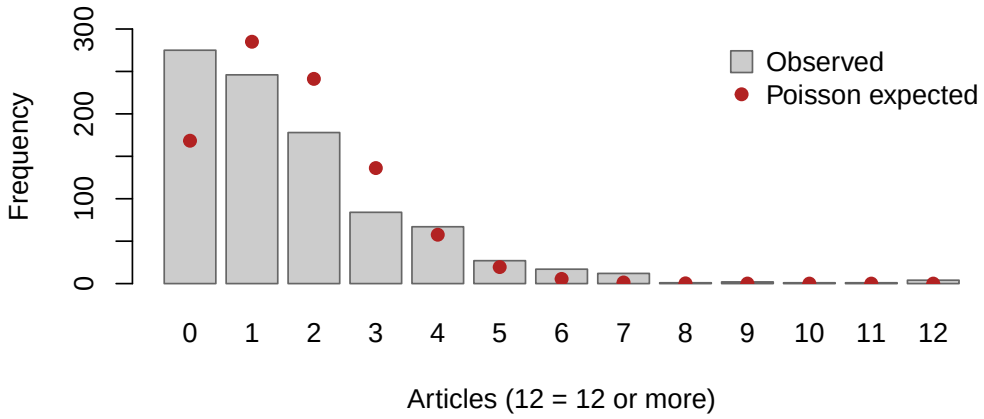


Figure 1: Observed frequencies of article counts (bars) and frequencies expected under a Poisson distribution with the same mean (points).

A.2 Model fitting

```
m_pois <- glm(art ~ fem + mar + kid5 + phd + ment, family = poisson, data = d)
m_nb   <- glm.nb(art ~ fem + mar + kid5 + phd + ment, data = d)
m_zinb <- zeroinfl(art ~ fem + mar + kid5 + phd + ment |
                         fem + mar + kid5 + phd + ment,
                   data = d, dist = "negbin")
summary(m_nb)$coefficients
```

```
#>                Estimate  Std. Error    z value     Pr(>|z|)
#> (Intercept)  0.25614402 0.137347701  1.8649313 6.219104e-02
#> femWomen    -0.21641842 0.072636088 -2.9794890 2.887296e-03
#> marMarried   0.15048945 0.082096543  1.8330790 6.679082e-02
#> kid5        -0.17641524 0.052813298 -3.3403565 8.367091e-04
#> phd          0.01527116 0.035872991  0.4257006 6.703260e-01
#> ment         0.02908234 0.003214319  9.0477454 1.459506e-19
```

```
summary(m_zinb)
```

```
#>
#> Call:
#> zeroinfl(formula = art ~ fem + mar + kid5 + phd + ment | fem + mar +
#>     kid5 + phd + ment, data = d, dist = "negbin")
#>
#> Pearson residuals:
#>     Min      1Q  Median      3Q     Max
#> -1.2942 -0.7601 -0.2909  0.4448  6.4155
#>
#> Count model coefficients (negbin with log link):
```

```
#>               Estimate Std. Error z value Pr(>|z|)
#> (Intercept)  0.4167465  0.1435966   2.902  0.00371 **
#> femWomen    -0.1955068  0.0755926  -2.586  0.00970 **
#> marMarried   0.0975826  0.0844520   1.155  0.24789
#> kid5        -0.1517325  0.0542061  -2.799  0.00512 **
#> phd         -0.0007001  0.0362697  -0.019  0.98460
#> ment         0.0247862  0.0034927   7.097 1.28e-12 ***
#> Log(theta)   0.9763565  0.1354695   7.207 5.71e-13 ***
#>
#> Zero-inflation model coefficients (binomial with logit link):
#>             Estimate Std. Error z value Pr(>|z|)
#> (Intercept) -0.19169    1.32282  -0.145  0.88478
#> femWomen     0.63593    0.84892   0.749  0.45379
#> marMarried  -1.49947    0.93867  -1.597  0.11017
#> kid5         0.62843    0.44278   1.419  0.15582
#> phd         -0.03771    0.30801  -0.122  0.90254
#> ment        -0.88229    0.31623  -2.790  0.00527 **
#> ---
#> Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
#>
#> Theta = 2.6548
#> Number of iterations in BFGS optimization: 43
#> Log-likelihood: -1550 on 13 Df
```

A.3 Model comparison

```
exp_zero <- c(
  Poisson = sum(dpois(0, fitted(m_pois))),
  NB      = sum(dnbinom(0, size = m_nb$theta, mu = fitted(m_nb))),
  ZINB    = sum(predict(m_zinb, type = "prob")[, 1]))
cmp <- data.frame(
  Model = names(exp_zero),
  df = c(attr(logLik(m_pois), "df"), attr(logLik(m_nb), "df"),
         attr(logLik(m_zinb), "df")),
  logLik = c(logLik(m_pois), logLik(m_nb), logLik(m_zinb)),
  AIC = c(AIC(m_pois), AIC(m_nb), AIC(m_zinb)),
  BIC = c(BIC(m_pois), BIC(m_nb), BIC(m_zinb)),
  ExpectedZeros = exp_zero)
knitr::kable(cmp, digits = 1, row.names = FALSE, booktabs = TRUE)
```

Table 7: Model comparison for the bioChemists data (observed zeros = 275).

Model	df	logLik	AIC	BIC	ExpectedZeros
Poisson	6	-1651.1	3314.1	3343.0	191.4
NB	7	-1561.0	3135.9	3169.6	277.8
ZINB	13	-1550.0	3126.0	3188.6	285.4

The Poisson model predicts only about 191 zeros, against 275 observed, and fits much worse than both alternatives (AIC higher by about 178). Allowing overdispersion fixes most of the problem. The NB model alone predicts about 278 zeros, which is **very close to the 275 observed**. So the "excess zeros" relative to the Poisson are largely explained by overdispersion. The ZINB model improves the log-likelihood by 11 with 6 extra parameters. **AIC slightly favors ZINB** (3126 vs. 3135.9), while **BIC favors NB** (3169.6 vs. 3188.6).

This is an important lesson: **more zeros than a Poisson predicts does not by itself justify a zero-inflated model.** Always compare against the NB model. (In NMES1988, by contrast, the NB model under-predicts the zeros noticeably.)

A.4 Interpretation

```
wald <- function(est, se) {
  sprintf("%.2f (%.2f, %.2f)", exp(est),
          exp(est - 1.96 * se), exp(est + 1.96 * se))
}
s_nb <- summary(m_nb)$coefficients[-1, ]
s_zc <- summary(m_zinb)$coefficients$count
s_zc <- s_zc[!(rownames(s_zc) %in% c("(Intercept)", "Log(theta)")), ]
s_zz <- summary(m_zinb)$coefficients$zero[-1, ]
```

```
knitr::kable(data.frame(Covariate = rownames(s_nb),
                        `NB IRR (95% CI)` = wald(s_nb[, 1], s_nb[, 2]),
                        `ZINB count IRR (95% CI)` = wald(s_zc[, 1], s_zc[, 2]),
                        check.names = FALSE),
             row.names = FALSE, booktabs = TRUE, align = "lcc")
```

Table 8: Count part: incidence rate ratios (IRR) with 95% Wald confidence intervals.

Covariate	NB IRR (95% CI)	ZINB count IRR (95% CI)
femWomen	0.81 (0.70, 0.93)	0.82 (0.71, 0.95)
marMarried	1.16 (0.99, 1.37)	1.10 (0.93, 1.30)
kid5	0.84 (0.76, 0.93)	0.86 (0.77, 0.96)
phd	1.02 (0.95, 1.09)	1.00 (0.93, 1.07)
ment	1.03 (1.02, 1.04)	1.03 (1.02, 1.03)

```
knitr::kable(data.frame(Covariate = rownames(s_zz),
                        `OR (95% CI)` = wald(s_zz[, 1], s_zz[, 2]),
                        `p-value` = format.pval(s_zz[, 4], digits = 2),
                        check.names = FALSE),
             row.names = FALSE, booktabs = TRUE, align = "lcc")
```

Table 9: ZINB zero part: odds ratios (OR) of being a structural zero, with 95% Wald confidence intervals.

Covariate	OR (95% CI)	p-value
femWomen	1.89 (0.36, 9.97)	0.4538
marMarried	0.22 (0.04, 1.41)	0.1102
kid5	1.87 (0.79, 4.47)	0.1558
phd	0.96 (0.53, 1.76)	0.9025
ment	0.41 (0.22, 0.77)	0.0053

Dispersion. The NB model estimates $\hat{\theta} = 2.26$ (SE 0.27), so the variance at the mean, $\mu + \mu^2/\theta$, is about 2.96, compared with a Poisson variance of 1.69.

NB model. Holding other variables fixed:

- **Mentor productivity** is the most precisely estimated effect ($z \approx 9$). Each additional mentor article multiplies the expected number of articles by 1.03 (about 3% more). Ten more mentor articles multiply it by 1.34.
- **Women** are expected to publish 19% fewer articles than men (IRR = 0.81).
- **Each young child** lowers the expected count by about 16% (IRR = 0.84).
- Marital status and program prestige are not significant at the 5% level.

ZINB model, count part. This part describes students who are "at risk" of publishing. The conclusions match the NB model. Gender (IRR = 0.82), young children (IRR = 0.86), and mentor productivity (IRR = 1.025 per mentor article) are significant, with slightly smaller effect sizes because some zeros are now attributed to the zero component.

ZINB model, zero part. Only **mentor productivity** is significant. Each additional mentor article multiplies the odds of being a structural (never-publishing) zero by 0.41, a 59% reduction. Students whose mentors are productive are therefore less likely to belong to the group that does not publish at all, *and* they publish more if they do publish. The zero-part standard errors are large (e.g., the intercept SE is about 1.3), which shows that the zero component is weakly identified in these data. This is consistent with BIC preferring the simpler NB model.

Bottom line. For `bioChemists`, the NB model is an adequate and more parsimonious description; ZINB offers a modest improvement in AIC and a useful two-process interpretation for mentor productivity. Your NMES1988 report should reach and defend its own conclusion in the same way.